Project Phase 2 — Statistical Damage Detection

Course: CIV E 789 Sensing Techniques and Data Analytics (Fall 2026)
Instructor: Mohammad Talebi-Kalaleh, Ph.D.
Group: 6 (sensors S2, S4, S6, S9, S12, S15, S18, S21, S24)
Data: `phase2_modal_parameters.json` (intact, damage1, damage2; 20 estimates each)

Contents

0. Setup
1. Part A: Feature Distribution Analysis
2. Part B: Single Feature Damage Detection
3. Part C: Bivariate Analysis
4. Part D: Feature Fusion and Engineering
5. Part E: Principal Component Analysis
6. Part F: Engineering Insights
7. Use of Generative AI

0. Setup

This section loads the data used throughout the project and sets up the common tools. Parts A–E start from the arrays created here, so that all of them work with exactly the same data.

- **0.1** Libraries, paths and helper functions to present tables and figures.
- **0.2** Loading the JSON file and the group 6 sensors.
- **0.3** Extraction of natural frequencies and mode shapes, with automatic checks.

0.1 Libraries, paths and presentation helpers

- The file `phase2_modal_parameters.json` is searched for first in the notebook folder and, if it is not there, in the parent folder.
- Figures are saved in `figures/part-a/` … `figures/part-e/` and result tables in `results/` (CSV). The folders are created if they do not exist.
- `show_table` places the table caption above the table (**Table N.**) and `show_figure` places the figure caption below the figure (**Figure N.**), both outside the object. `show_figure` also saves the figure in the folder of its part.

```
import json
from pathlib import Path

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy import stats
from IPython.display import display, Markdown

# Input data: next to this notebook or, if not there, in the parent folder
```

```
DATA_FILE = 'phase2_modal_parameters.json'
DATA_PATH = next((p for p in (Path(DATA_FILE), Path('..') / DATA_FILE) if p.exists()),
None)
assert DATA_PATH is not None, f'{DATA_FILE} not found next to the notebook or in its
parent folder'

# Output folders: one figure folder per part, one folder for result tables
FIG_DIR = Path('figures')
RES_DIR = Path('results')
for part in ['part-a', 'part-b', 'part-c', 'part-d', 'part-e']:
    (FIG_DIR / part).mkdir(parents=True, exist_ok=True)
RES_DIR.mkdir(exist_ok=True)


def show_table(df, number, caption, filename=None, precision=4):
    """Show the table caption above the table; optionally save the table as CSV in
results/."""
    display(Markdown(f'**Table {number}.** {caption}'))
    display(df.style.format(precision=precision))
    if filename is not None:
        df.to_csv(RES_DIR / filename)


def show_figure(fig, number, caption, part, filename):
    """Save the figure in figures/<part>/, show it and put the caption below it."""
    fig.savefig(FIG_DIR / part / filename, dpi=300, bbox_inches='tight')
    display(fig)
    plt.close(fig)
    display(Markdown(f'**Figure {number}.** {caption}'))
```

0.2 Data loading and group 6 sensors

- The JSON file has two blocks: `scenarios` (data for `intact`, `damage1` and `damage2`) and `metadata` (sampling information, sensors assigned to each group and position of each sensor).
- The group sensors are read from `metadata.sensor_assignments.group_6` instead of being typed by hand, to avoid copying errors.
- Each mode shape is a list of 24 values ordered from S1 to S24, so sensor S_k is at index $k-1$ of the list (Table 1).
- The position of each sensor (`metadata.sensor_positions_m`) will be used as the x-axis of the mode shapes.

```
with open(DATA_PATH, 'r') as f:
    data = json.load(f)

GROUP = 6
SCENARIOS = ['intact', 'damage1', 'damage2']
MODES = ['mode_1', 'mode_2', 'mode_3']
N_MEAS = data['metadata']['sampling_info']['total_estimates']   # 20 estimates per
condition
N_SENSORS_TOTAL = len(data['metadata']['sensor_positions_m'])   # 24 sensors on the
bridge

# Sensors assigned to our group, read from the file (not typed by hand)
```

```
my_sensors = data['metadata']['sensor_assignments'][f'group_{GROUP}']
SENSOR_IDX = np.array([int(s[1:]) - 1 for s in my_sensors])   # Sk -> list index k - 1
SENSOR_POS = np.array([data['metadata']['sensor_positions_m'][s] for s in my_sensors])
# [m]

sensors_table = pd.DataFrame({'List index': SENSOR_IDX, 'Position (m)': SENSOR_POS},
                             index=pd.Index(my_sensors, name='Sensor'))
show_table(sensors_table, 1, f'Sensors assigned to group {GROUP}, their index in the
24-value lists '
           'and their position along the bridge.', precision=1)
```

Table 1. Sensors assigned to group 6, their index in the 24-value lists and their position along the bridge.

Sensor	List index	Position (m)
S2	1	7.5
S4	3	17.5
S6	5	27.5
S9	8	42.5
S12	11	57.5
S15	14	72.5
S18	17	87.5
S21	20	102.5
S24	23	117.5

0.3 Extraction of natural frequencies and mode shapes

The first cell defines the two extraction functions from the assignment starter code (Section 7), with the same names and behaviour: `extract_frequencies(scenario, mode_key)` returns the 20 frequencies of one mode, and `extract_mode_shape(scenario, measurement_idx, mode_key)` returns one mode shape at the 9 group sensors. The second cell uses them to store the data in numpy arrays, one per scenario (Table 2).

Table 2. Numpy arrays created in the data extraction.

Array	Size	Content
`freqs[scenario]`	20 × 3	measurements × modes, natural frequency in Hz
`shapes[scenario]`	3 × 20 × 9	modes × measurements × group 6 sensors

- **Assumption:** value i of each frequency list and `measurement_i` of the mode shapes come from the same 30-s segment. The assignment does not state this explicitly. It does not matter for Parts A and B, which analyse each feature separately, but Parts C, D and E combine several features of the same measurement into a single point.

- Measurements are read as `measurement_1` … `measurement_20` in numeric order, not in the order of the JSON keys.
- Mode shapes are not normalized again: they are already scaled to a maximum of 1 over the 24 sensors. The MAC does not depend on scale, so this does not affect it.
- The checks (`assert`) stop the notebook if a measurement is missing, if a mode shape does not have 24 values, or if there are missing values (NaN).

```
# Extraction functions from the starter code (same names and behaviour)

# Extract frequencies for a scenario
def extract_frequencies(scenario, mode_key='mode_1'):
    return np.array(data['scenarios'][scenario]['natural_frequencies'][mode_key])


# Extract mode shape at assigned sensors
def extract_mode_shape(scenario, measurement_idx, mode_key='mode_1'):
    mode_data = data['scenarios'][scenario]['mode_shapes'][mode_key]
[f'measurement_{measurement_idx}']
    # mode_data is a list of 24 values ordered S1..S24, so S1 -> index 0
    return np.array([mode_data[int(s[1:]) - 1] for s in my_sensors])
```

```
# freqs[scenario]:  (N_MEAS, 3) array -> measurements x modes, in Hz
# shapes[scenario]: (3, N_MEAS, 9) array -> modes x measurements x our 9 sensors
# Measurements are read as measurement_1 ... measurement_20 (numeric order), so row i
of
# freqs and measurement i of shapes refer to the same 30-s segment.
freqs = {}
shapes = {}
for sc in SCENARIOS:
    freqs[sc] = np.column_stack([extract_frequencies(sc, m) for m in MODES])
    shapes[sc] = np.array([[extract_mode_shape(sc, i, m) for i in range(1, N_MEAS +
1)]
                           for m in MODES])

# Sanity checks on the raw file and on the extracted arrays
for sc in SCENARIOS:
    for m in MODES:
        assert len(data['scenarios'][sc]['natural_frequencies'][m]) == N_MEAS
        mode_shapes = data['scenarios'][sc]['mode_shapes'][m]
        assert set(mode_shapes) == {f'measurement_{i}' for i in range(1, N_MEAS + 1)}
        assert all(len(v) == N_SENSORS_TOTAL for v in mode_shapes.values())
    assert freqs[sc].shape == (N_MEAS, len(MODES))
    assert shapes[sc].shape == (len(MODES), N_MEAS, len(my_sensors))
    assert np.isfinite(freqs[sc]).all() and np.isfinite(shapes[sc]).all()
print('Extraction checks passed: '
      f'{len(SCENARIOS)} scenarios x {len(MODES)} modes x {N_MEAS} measurements x
{len(my_sensors)} sensors')
```

```
Extraction checks passed: 3 scenarios x 3 modes x 20 measurements x 9 sensors
```

```
# One table per scenario: rows = modes, columns = measurements
for t, sc in enumerate(SCENARIOS, start=3):
    freq_table = pd.DataFrame(freqs[sc].T,
                              index=pd.Index([f'f{k + 1} (Hz)' for k in
range(len(MODES))], name='Mode'),
```

```
                                columns=pd.RangeIndex(1, N_MEAS + 1,
name='Measurement'))
    show_table(freq_table, t, f'Natural frequencies of modes 1–3 for each of the 20
measurements, {sc} scenario.',
               precision=3)
```

Table 3. Natural frequencies of modes 1–3 for each of the 20 measurements, intact scenario.

Measurement	1	2	3	4	5	6	…	16	17	18	19	20
Mode												
f1 (Hz)	1.063	1.274	1.259	0.973	1.154	1.276	…	1.091	1.095	1.091	1.327	1.402
f2 (Hz)	2.847	3.404	3.253	2.537	3.052	3.345	…	2.875	2.801	2.855	3.450	3.733
f3 (Hz)	4.083	4.945	4.795	3.677	4.330	4.912	…	4.085	4.164	4.126	4.961	5.368

Table 4. Natural frequencies of modes 1–3 for each of the 20 measurements, damage1 scenario.

Measurement	1	2	3	4	5	6	…	16	17	18	19	20
Mode												
f1 (Hz)	1.131	1.191	1.262	1.052	0.994	1.232	…	1.098	1.154	1.082	0.988	0.966
f2 (Hz)	2.777	2.962	3.067	2.769	2.429	3.077	…	2.778	2.852	2.790	2.549	2.569
f3 (Hz)	4.284	4.506	4.717	4.059	3.738	4.581	…	4.026	4.319	4.054	3.708	3.906

Table 5. Natural frequencies of modes 1–3 for each of the 20 measurements, damage2 scenario.

Measurement	1	2	3	4	5	6	…	16	17	18	19	20
Mode												
f1 (Hz)	1.212	1.052	1.095	1.054	1.064	1.035	…	0.981	1.159	1.096	0.917	1.255
f2 (Hz)	2.793	2.557	2.841	2.533	2.621	2.488	…	2.555	2.955	2.681	2.338	3.016
f3 (Hz)	4.491	4.064	4.379	3.992	3.980	4.107	…	3.802	4.416	4.161	3.572	4.662

Part A: Feature Distribution Analysis (25 pts)

A1. Natural Frequency Distributions (8 pts)

A2. Mode Shape Visualization (9 pts)

A3. Modal Assurance Criterion (MAC) Distributions (8 pts)

Part B: Single Feature Damage Detection (20 pts)

B1. Z-Score Anomaly Detection (10 pts)

B2. Wasserstein Distance Analysis (7 pts)

B3. Feature Sensitivity Ranking (3 pts)

Part C: Bivariate Analysis (20 pts)

C1. Joint Distribution Visualization (7 pts)

C2. Mahalanobis Distance Detection (8 pts)

C3. Bivariate vs Univariate Comparison (5 pts)

Part D: Feature Fusion and Engineering (15 pts)

D1. Systematic Combination Testing (10 pts)

D2. Feature Importance Analysis (5 pts)

Part E: Principal Component Analysis (15 pts)

E1. PCA Implementation and Variance Analysis (4 pts)

E2. First Principal Component Analysis (3 pts)

E3. Two-Dimensional PCA Space (3 pts)

E4. Progressive Dimensionality Impact (3 pts)

E5. Physical Interpretation (2 pts)

Part F: Engineering Insights (5 pts)

F1. Method Comparison and Selection (2 pts)

F2. Feature Selection Strategy (2 pts)

F3. Practical Implementation Considerations (1 pt)

Use of Generative AI